Развёртывание и сопровождение

Docker / Linux

Требуются Docker Engine, Compose v2, доступ к интернету для первой сборки, ориентировочно 4 CPU, 8 ГБ RAM и 8 ГБ диска. Рекомендации по ресурсам — стартовая конфигурация, а не подтверждённая граница мощности.

```
cp .env.example .env
```

```
docker compose config
```

```
docker compose up --build -d
```

```
docker compose ps
```

```
docker compose logs -f backend
```

Windows PowerShell: Copy-Item .env.example .env. Используйте Docker Desktop с Linux containers. Адреса: frontend localhost:3000, backend localhost:8000, Swagger localhost:8000/docs, Keycloak localhost:8080. Остановить без потери данных: docker compose down.

На Windows можно запустить start.bat или powershell -File scripts/verify-compose.ps1: сценарий проверит Engine и Compose config, соберёт сервисы, дождётся healthchecks, проверит реальный Redis PING и Alembic. Он не устанавливает Docker/WSL и не меняет компоненты Windows. Перед Compose остановите локальные native-сервисы, занимающие те же порты.

FRONTEND_PORT, BACKEND_PORT, KEYCLOAK_PORT меняют опубликованные порты. При смене frontend-порта также измените redirectUris и webOrigins клиента crm-frontend в docker/keycloak-realm.json до первого запуска, либо в уже работающем realm через консоль Keycloak. Realm import не перезаписывает существующий realm при перезапуске. При смене hostname согласуйте KC_HOSTNAME, KEYCLOAK_ISSUER, redirect URI и CORS. Токен с issuer localhost не подходит для другого домена.

PostgreSQL init SQL создаёт схему keycloak только при инициализации пустого volume. Для старой БД выполните CREATE SCHEMA IF NOT EXISTS keycloak AUTHORIZATION crm;. Файлы живут в uploads, БД — postgres_data, Redis — redis_data. MAX_UPLOAD_MB ограничивает backend; если увеличиваете лимит выше 20 МБ, согласуйте client_max_body_size в docker/nginx.conf.

Для POSTGRES_PASSWORD в текущем Compose используйте случайную URL-safe строку (например, 32+ латинских букв и цифр): пароль включён в DATABASE_URL. Если нужны специальные символы, задайте отдельный URL с percent-encoding и исходный пароль PostgreSQL/Keycloak отдельно.

Миграции

```
docker compose exec backend alembic current
```

```
docker compose exec backend alembic upgrade head
```

```
docker compose exec backend alembic downgrade -1
```

Создавайте миграции в рабочей копии, где есть модели и доступ к БД:

```
cd backend
```

```
alembic revision --autogenerate -m "describe_change"
```

```
alembic upgrade head
```

```
alembic downgrade -1
```

Всегда просматривайте сгенерированный Python и проверяйте downgrade на отдельной БД. Первичный downgrade до base удаляет таблицы CRM: перед этим нужен backup. Изменять базу Keycloak миграциями CRM нельзя. Production startup не вызывает create_all.

Разработка без Docker

На текущем подготовленном Windows-стенде запустите start-local.bat. Он использует существующие .runtime/pgsql, .runtime/pgdata, .runtime/jdk-21*, .runtime/keycloak-26.2.5, .tools и node_modules; не скачивает программы и не меняет компоненты Windows. Все процессы запускаются скрыто, логи — .runtime/logs. Сценарий применяет миграции, сохраняет данные, проверяет issuer Keycloak и frontend API proxy. Он предназначен для этой демонстрационной БД; не направляйте его на production PostgreSQL. Redis необязателен для работы CRM: при отсутствии сервис продолжает работу без кэша.

Эти portable runtime не распространяются в репозитории. Для новой машины настройте сервисы по следующей инструкции или используйте Compose на Linux/Docker-хосте. Docker Compose здесь подготовлен, но интеграционный запуск контейнеров на данной Windows-машине не выполнялся из-за системного ограничения.

Поднимите PostgreSQL 17, Redis 7 и Keycloak 26.2.5, импортируйте realm. Python 3.12+ и Node 22.12+/24. Создайте virtualenv, установите backend/requirements-dev.txt. Из корня задайте PYTHONPATH=backend, DATABASE_URL и остальные переменные из .env.example; выполните Alembic и python -m app.db.seed, затем uvicorn app.main:app --host 127.0.0.1 --port 8000. npm ci и npm run dev запускают frontend на 3000. В Windows при блокировке npm.ps1 используйте npm.cmd.

Проверки

```
pytest backend/tests -q
```

```
ruff check backend/app backend/alembic backend/tests
```

```
ruff format --check backend/app backend/alembic backend/tests
```

```
npm test
```

```
npm run lint
```

```
npm run typecheck
```

```
npm run build
```

По умолчанию backend-тесты работают на изолированной SQLite БД. Для проверки PostgreSQL создайте ОТДЕЛЬНУЮ пустую БД и задайте TEST_DATABASE_URL. Тесты выполняют downgrade до base и уничтожают свои тестовые данные; никогда не передавайте рабочую БД. node scripts/browser-smoke.mjs использует локальный Edge, настоящий Keycloak и запущенный стенд. node scripts/load-check.mjs выполняет 50 параллельных запросов и 10 отчётов; результаты записываются в docs.

Более строгий сценарий: python scripts/load_test.py --users 50 --rounds 5. Он создаёт 50 независимых OIDC/PKCE-сессий, делает 250 запросов и 10 отчётов, затем завершает сессии. LOAD_USERNAME/LOAD_PASSWORD переопределяют демоучётку; секреты остаются в памяти процесса. Результат — docs/load-sessions-results.json. Тест предназначен для демонстрационного/тестового стенда и создаёт записи аудита экспорта.

Масштабирование

В учебном стенде используется один Uvicorn worker, пул SQLAlchemy 20 соединений + 40 overflow, синхронные отчёты в thread pool. Размер выборки отчёта ограничен 50 000, импорта — 10 000 строк. Для тяжёлых производственных объёмов вынесите генерацию в очередь workers с ограничением конкурентности и сохранением результатов. Нагрузочный тест на демонстрационных данных не

является SLA для реальных объёмов. Следите за длительностью SQL, CPU, памятью, ожиданием пула и cache hit rate.